

Introduction — A Readable Companion to the OTP Docs

Learning Erlang from Claude · checked against OTP 29.1.1 · Sep 25, 2026

Why this book exists

AI assistants now write a large share of everyday code. They are very good at the parts a traditional textbook spends most pages on: syntax, library signatures, boilerplate. What they don't reliably get right is how the runtime actually behaves: what happens to a message nobody matches, which process owns a table, when a binary is copied, what a timeout really means.

That shifts what a programmer needs to know. Less reference detail, but not less *mechanism* detail. You can only judge generated code as well as you understand what runs underneath it. This book is built on that idea.

The official Erlang/OTP documentation is accurate and current, but written as reference: precise, spread across many guides, and hard to read from start to finish. This companion reorganises it around concepts and mechanisms, and adds the one thing reference documentation never includes: code that looks right and isn't.

How each chapter is built

Every chapter has the same five parts:

1. **Why it exists.** The concept, and the problem it solves.
2. **The building blocks.** The API, condensed into tables.
3. **How the runtime actually does it.** The mechanics, from the Reference Manual and Efficiency Guide.
4. **Plausible but wrong.** Code that compiles and passes a happy-path test, why it fails under crashes, timing or load, and the fix.
5. **Review checklist and sources.** What to check in real code, with links to the docs it came from.

Chapter 13 adds a worked example before the runtime section, so its “Plausible but wrong” part is section 5. The cheat sheet (chapter 14) follows its own layout.

The “Plausible but wrong” section is the heart of each chapter. Every example there is the kind of code an assistant (or a tired human) produces readily. Learning to spot them is the skill this book is trying to build.

How to read it

The chapters build on each other, but not strictly.

Part	Chapters	Read when
Foundations	1 Supervision Trees · 2 Processes and Mailboxes	First, in either order
Data	3 Binaries and Memory · 4 ETS	After chapter 2
Building systems	5 <code>gen_server</code> and <code>gen_statem</code> · 6 Distribution · 7 Applications and Releases	After the foundations
Keeping it working	8 Testing · 9 Observability · 10 Schedulers and Performance	Any time after chapter 5
Edges	11 NIFs and Ports · 12 Elixir Alongside Erlang	When you need them
Deep dive	13 <code>gen_statem</code> in Depth	After chapter 5
Reference	14 Review Cheat Sheet	When reviewing code

If you know Elixir but not Erlang, start with chapter 12 section 2 for the mapping between the two, then read from chapter 1. Everything about the runtime applies to both languages unchanged.

Using it with an AI assistant

Some ways this book is meant to be used:

- **As a review list.** Chapter 14 collects every checklist. Run generated code past it before accepting it.
- **As a prompt.** Ask the assistant to check its own code against a specific chapter’s “Plausible but wrong” section, or to write a test that would expose a given failure (chapter 8 shows how).
- **As a check on the assistant.** When an assistant claims something about runtime behaviour, the sources at the end of each chapter point to where you can verify it.

Conventions

- Memory sizes are in *words*: 8 bytes on a 64-bit system.
- Facts were checked against the Erlang/OTP 29.1.1 documentation, and the Elixir documentation for chapter 12. Version-specific behaviour is marked where it matters (for example “since OTP 24”).
- Code examples are deliberately short. They show one mistake each, not complete programs.

A note on accuracy

This book was written with Claude, an AI assistant, and every chapter was checked against the official documentation it cites. It can still contain mistakes, which would be fitting given its subject. Treat the linked documentation as the authority, and please report errors as issues on the repository.